

HỆ THỐNG AI QUẢN LÝ NHÀ TRỢ
Kiến trúc Router-Centric ReAct (Dual-Process)

TroManager

Architecture #2

Báo cáo Kỹ thuật

Thiết kế, Triển khai và Đánh giá

Thông tin dự án

- Dự án:** TroManager Architecture #2
Lĩnh vực: Hệ thống AI đa tác tử
Ứng dụng: Quản lý và chăm sóc khách hàng nhà trọ

Tóm tắt

Báo cáo này trình bày **TroManager Architecture #2**, một hệ thống AI đa tác tử cho bài toán quản lý và chăm sóc khách hàng trong mô hình kinh doanh phòng trọ. Hệ thống áp dụng kiến trúc *Router-Centric Dual-Process* dựa trên Lý thuyết Nhận thức Kép của Daniel Kahneman, kết hợp hai luồng xử lý: System 1 (phản xạ nhanh, chi phí thấp) và System 2 (lý luận chậm, phức tạp).

Ý tưởng cốt lõi là phân tách request thành hai lớp chi phí: **System 1** sử dụng mô hình flash (gemma-4-31b-it) kết hợp semantic cache (pgvector HNSW, cosine >0.9) và RAG (LlamaIndex, 18 tài liệu) để xử lý 80% câu hỏi đơn giản; **System 2** sử dụng mô hình pro (gemini-3.1-flash-lite) với vòng lặp ReAct (tối đa 4 bước) và Dynamic Tool Registry (18 tools, 3 toolkits) cho các tác vụ phức tạp như tài chính, hợp đồng, và tương tác nhạy cảm. Một **Router Gateway** dựa trên LLM (gemma-4-26b-a4b-it) quyết định luồng xử lý cho mỗi request.

Hệ thống được triển khai với FastAPI (Python 3.13), PostgreSQL 16 + pgvector (3072 chiều), bảy cron jobs cho tương tác chủ động (invoice overdue, contract expiry, birthday, maintenance), cơ chế Human-in-the-loop qua Approval Queue, và Persona Optimizer trigger-based cho cá nhân hóa động. Kết quả đánh giá trên 20 kịch bản mô phỏng đa lượt cho thấy khả năng xử lý chính xác các tình huống thực tế, từ tra cứu hóa đơn, bảo trì, gia hạn hợp đồng đến xử lý khiếu nại và tư vấn phòng trống.

Từ khóa: Hệ thống đa tác tử, Dual-Process Architecture, Router-Centric ReAct, AI quản lý nhà trọ, Retrieval-Augmented Generation, PostgreSQL pgvector, FastAPI.

Mục lục

Tóm tắt	1
1 Giới thiệu	4
1.1 Mục tiêu thiết kế	4
1.2 Đóng góp chính	4
1.3 Cấu trúc báo cáo	4
2 Kiến trúc hệ thống	5
2.1 Thiết kế tổng thể	5
2.2 Luồng xử lý request	6
2.3 Công nghệ sử dụng	6
3 LLM Intent Router	7
3.1 Thiết kế	7
3.2 Ánh xạ Intent	8
4 System 1: Fast Layer	9
4.1 Pipeline	9
4.2 Semantic Cache	9
5 System 2: ReAct Agent	10
5.1 Kiến trúc	10
5.2 State Machine	10
5.3 Context Injection	10
5.4 Vòng lặp ReAct	12
5.5 Timeout Protection	12
5.6 Guardrails	12
5.7 Multi-Turn Conversation	12
6 Tool Registry	13
6.1 Dynamic Loading	13
6.2 Decision Toolkit (5 tools)	13
6.3 Knowledge Toolkit (7 tools)	14
6.4 Automation Toolkit (6 tools)	14
7 User Modeling & Personalisation	15
7.1 Kiến trúc	15
7.2 Persona Optimizer	15
7.3 Core Memory Updates	16
7.4 Approval Service	16

8	Proactive System	17
8.1	Bây Cron Jobs	17
8.2	Event Dispatcher	17
8.3	Anti-Spam Guard	17
8.4	Golden Time Delivery	18
9	Database Schema	19
9.1	Entity Relationship Diagram	19
9.2	Danh sách bảng	20
9.3	SQL Views & Indexing	20
10	Observability & API	21
10.1	JSON Structured Logging	21
10.2	Prometheus Metrics	21
10.3	API Endpoints	21
10.4	Bảo mật	22
11	Cấu hình & Triển khai	23
11.1	Hệ thống Config hai tầng	23
11.2	Quick Start	23
12	Đánh giá	24
12.1	Kịch bản đánh giá	24
12.2	Kết quả	24
12.3	Phân tích độ trễ	24
13	Tổng kết & Lộ trình	25
13.1	Tổng kết	25
13.1.1	Các luận điểm chính	25
13.1.2	Hiện trạng	26
13.2	Lộ trình tương lai	26
13.3	Key Source Files	27

Giới thiệu

1.1 Mục tiêu thiết kế

Sáu mục tiêu thiết kế được xác định:

1. **Chi phí thấp:** 80% câu hỏi đơn giản xử lý bởi luồng chi phí thấp (semantic cache + flash LLM).
2. **Cá nhân hóa:** Hồ sơ khách hàng động, cập nhật dựa trên tương tác thực tế.
3. **Chủ động:** Hệ thống tự động phát hiện và xử lý sự kiện (nợ, hết hạn, bảo trì, sinh nhật).
4. **An toàn:** Tác vụ nhạy cảm (gửi tin nhắn, nhắc nợ) qua Human-in-the-loop.
5. **Bảo mật:** Không rò rỉ thông tin giữa các tenant, webhook có HMAC-SHA256.
6. **Quan sát được:** Prometheus metrics, JSON structured logging, Request ID tracing.

1.2 Đóng góp chính

1. **Kiến trúc Router-Centric Dual-Process:** Phân tách request thành hai luồng chi phí với LLM Intent Router, khác biệt so với các hệ thống single-agent truyền thống.
2. **Semantic Cache trên pgvector:** Cache ngữ nghĩa với HNSW index (cosine >0.9), TTL 30 ngày, tự động warming với FAQ.
3. **Dynamic Tool Registry:** 18 tools được load linh hoạt theo intent, giảm thiểu context window cho agent.
4. **7 Cron Jobs chủ động:** Phát hiện nợ, hết hạn hợp đồng, bảo trì, sinh nhật với Anti-Spam Guard và Golden Time Delivery.
5. **Persona Optimizer trigger-based:** Chỉ chạy khi có tương tác mới, dùng flash model để tối ưu chi phí.

1.3 Cấu trúc báo cáo

Phần 2 trình bày kiến trúc tổng thể. Phần 3 mô tả LLM Intent Router. Phần 4 và 5 lần lượt trình bày System 1 và System 2. Phần 6 giới thiệu Tool Registry. Phần 7 thảo luận về User Modeling. Phần 8 trình bày hệ thống cron jobs. Phần 9 mô tả cơ sở dữ liệu. Phần 10 đề cập đến observability. Phần 12 báo cáo kết quả đánh giá. Phần 13 tổng kết và đề xuất hướng phát triển.

Kiến trúc hệ thống

2.1 Thiết kế tổng thể

TroManager Architecture #2 áp dụng mô hình **Router-Centric Dual-Process**, lấy cảm hứng từ Lý thuyết Nhận thức Kép (Dual-Process Theory) của Daniel Kahneman. Hệ thống gồm bốn thành phần chính:

- **Router Gateway:** LLM Intent Router (gemma-4-26b-a4b-it) với prompt yêu cầu tự phân loại System 1 hay System 2 dựa trên ngữ nghĩa toàn câu.
- **System 1 (Fast Layer):** gemma-4-31b-it + Semantic Cache (pgvector cosine >0.9) + RAG (LlamaIndex).
- **System 2 (ReAct Agent):** gemini-3.1-flash-lite + vòng lặp ReAct (tối đa 4 bước) + Dynamic Tool Registry.
- **User Modeling Layer:** Profile, Behavior, Memory, Persona Optimizer, Approval Service.

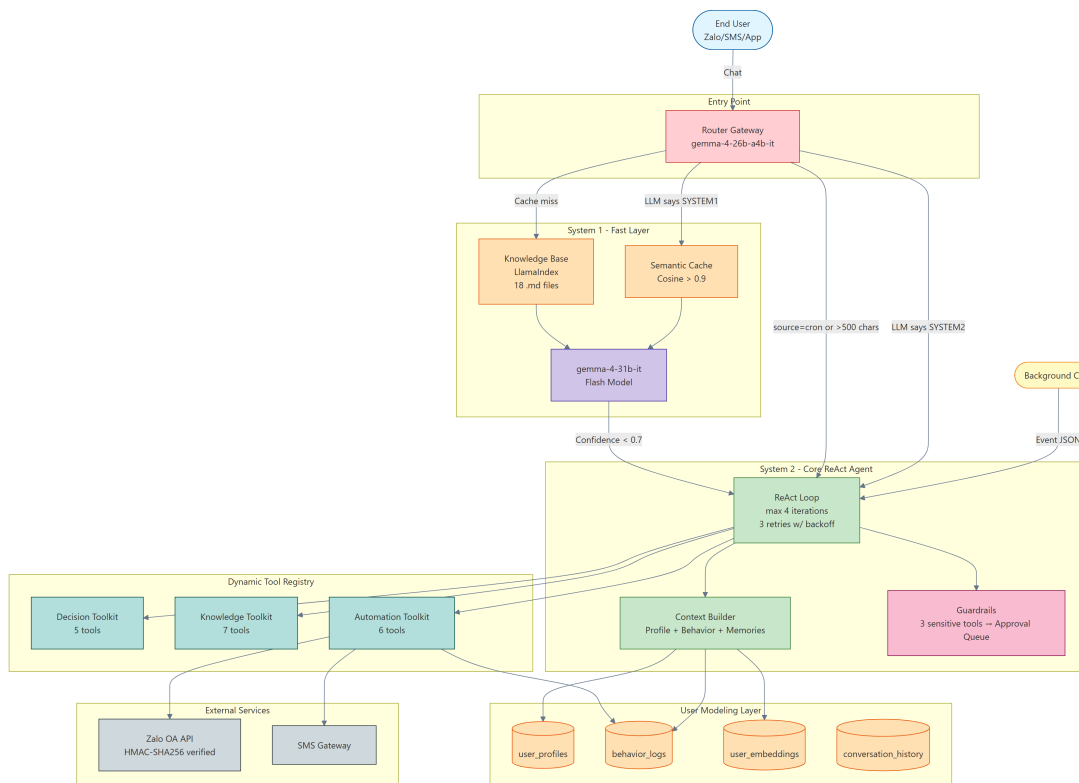


Figure 2.1: Kiến trúc tổng thể TroManager Architecture #2: Router Gateway nhận request, phân loại System 1 hay System 2, hai luồng xử lý song song với User Modeling Layer và Tool Registry.

2.2 Luồng xử lý request

Router Gateway thực hiện cây quyết định:

- Thuật toán route:
1. source == "cron" → System 2 (priority: HIGH)

2. Query dài > 500 ký tự → System 2 (priority: NORMAL)

3. Còn lại → LLMIntentRouter.classify()
 - LLM trả SYSTEM1 → System 1 (confidence threshold 0.9)
 - LLM trả SYSTEM2 → System 2
 - Timeout (5s) → System 2 (confidence 0.5)
 - Parse error → System 2 (confidence 0.5)

2.3 Công nghệ sử dụng

Thành phần	Công nghệ
Ngôn ngữ	Python 3.13
Web Framework	FastAPI
Database	PostgreSQL 16 + pgvector (HNSW, 3072 dims)
LLM Router	gemma-4-26b-a4b-it
LLM System 1	gemma-4-31b-it (Flash)
LLM System 2	gemini-3.1-flash-lite (Pro)
Embedding	gemini-embedding-2 (3072 dims)
Agent Runtime	LangChain + Custom ReAct loop
RAG	LlamaIndex trên 18 file .md
Notification	Zalo OA API + SMS Gateway
Background Jobs	APScheduler (AsyncIOScheduler)
Cấu hình	YAML + .env (python-dotenv)

LLM Intent Router

3.1 Thiết kế

Khác với các hệ thống rule-based truyền thống dùng keyword/entity pattern-matching, Router của TroManager sử dụng **LLM Intent Router** — gọi trực tiếp model gemma-4-26b-a4b-it với prompt phân loại ngữ nghĩa toàn câu. Cách tiếp cận này loại bỏ false positive từ pattern-matching cứng nhắc, đồng thời cho phép router hiểu được ngữ cảnh và sắc thái câu hỏi.

```
class LLMIntentRouter:
    def __init__(self, llm_client=None, config=None):
        self.llm = llm_client.with_model(
            llm_client.config.router_model
        )
        self.timeout = float(
            config.get("llm_timeout_seconds", 5.0)
        )

    async def classify(self, text: str) -> RouteMatch:
        prompt = (
            'You are a Router for an AI boarding house.\n'
            f'Read: "{text}"\n'
            'Decide: SYSTEM1 (simple/queries) / '
            'SYSTEM2 (complex/finance)\n'
            'Return JSON: {"system": "...", '
            '"confidence": ..., "reason": "...}'
        )
        response = await asyncio.wait_for(
            self.llm.generate(messages=[...], temperature=0.0),
            timeout=self.timeout
        )
```


3.2 Ánh xạ Intent

Intent	Toolkits	Hệ thống
billing_inquiry	Knowledge	S1
payment_reminder	Knowledge, Automation	S2
billing_dispute	Knowledge, Decision	S2
maintenance_request	Knowledge, Automation	S2
maintenance_status	Knowledge	S1
room_recommendation	Decision, Knowledge	S2
room_transfer	Decision, Knowledge, Automation	S2
room_inquiry	Decision, Knowledge	S2
contract_inquiry	Knowledge	S1
contract_renewal	Knowledge, Decision, Automation	S2
policy_question	Knowledge	S1
general_chat, greeting	(none)	S1
background_event_*	Knowledge, Automation	S2
unknown	Knowledge (safe)	S1

System 1: Fast Layer

4.1 Pipeline

System 1 (src/system1/fast_layer.py) xử lý câu hỏi qua pipeline 7 bước:

1. **Embedding:** gemini-embedding-2 (3072 chiều, timeout protection).
2. **Profile Context:** Inject tone_preference (professional/friendly/strict) từ user_profiles.
3. **Semantic Cache:** Tra pgvector HNSW (cosine > 0.9, TTL 30 ngày).
 - Hit → trả cached response ngay (không cần LLM).
 - Miss → tiếp bước 4.
4. **RAG Retrieval:** LlamaIndex, top_k=3, similarity threshold 0.5.
5. **LLM Generation:** gemma-4-31b-it, response_format=json_object.
6. **Confidence Check:** Nếu < 0.7, fallback sang System 2.
7. **Cache Persistence:** Nếu confidence > 0.85, lưu query-response vào cache.

4.2 Semantic Cache

Bảng semantic_cache lưu trữ các cặp câu hỏi-câu trả lời đã được embedding hóa:

```
CREATE TABLE semantic_cache (  
    cache_id          SERIAL PRIMARY KEY,  
    query_text        TEXT NOT NULL,  
    query_embedding    vector(3072) NOT NULL,  
    response_text      TEXT NOT NULL,  
    hit_count          INT DEFAULT 0,  
    expires_at         TIMESTAMP DEFAULT (CURRENT_TIMESTAMP + INTERVAL '30_days')  
);  
CREATE INDEX idx_cache_query_vector  
ON semantic_cache  
USING hnsw (query_embedding vector_cosine_ops);
```

Cache được khởi tạo với ba FAQ entries: mật khẩu wifi, giờ yên tĩnh, phí gửi xe. Cosine similarity threshold 0.9 đảm bảo chỉ match những câu hỏi có ngữ nghĩa gần như tương đương.

System 2: ReAct Agent

5.1 Kiến trúc

ReAct agent (src/system2/react_agent.py, 518 dòng) thực hiện vòng lặp reasoning-action với gemini-3.1-flash-lite qua LangChain:

```
class ReActAgent:
    def __init__(self, config, llm_client, context_builder,
                 guardrails, profile_service, behavior_tracker):
        self.llm = llm_client
        self.max_iterations = 4
        self.tool_timeout_seconds = 30.0
        self.llm_timeout_seconds = 360.0
```

5.2 State Machine

5.3 Context Injection

ContextBuilder (src/system2/context_builder.py) assemble context từ năm nguồn:

```
class ContextBuilder:
    async def build(self, tenant_id, query,
                   behavior_days=90, top_k_memories=3):
        profile = await self.profiles.get_profile(tenant_id)
        behavior = await self.behaviors.get_summary(
            tenant_id, days=behavior_days
        )
        memories = await self.memories.search_memories(
            tenant_id, query, top_k=top_k_memories
        )
        return AgentContext(tenant_id, profile, behavior, memories)
```

Behavior summary được inject vào System 2 prompt dạng:

```
Late payments: 2
On-time payments: 5
Repair requests: 1
Average delay: 3.5 days
Last interaction: 2026-06-10
```

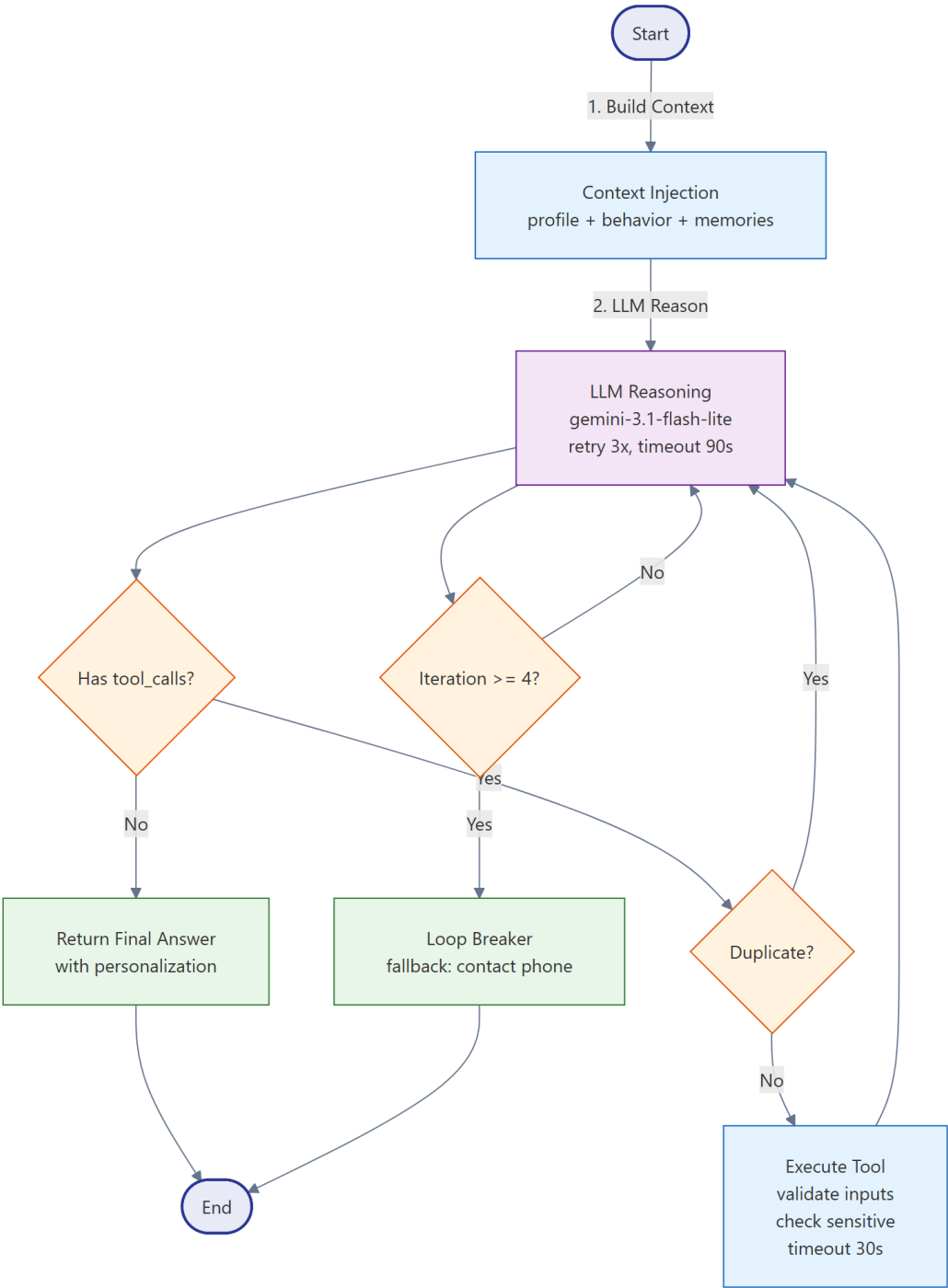


Figure 5.1: State machine của ReAct Agent: context injection, LLM reasoning, tool execution, duplicate detection, loop breaker.

5.4 Vòng lặp ReAct

1. **Inject context:** Profile, behavior summary, semantic memories, 5 conversation turns gần nhất.
2. **System Prompt:** Tên tenant, tone preference, thông tin hợp đồng, danh sách tool với mô tả.
3. **ReAct Loop** (tối đa 4 bước):
 - Token limit check: compress context nếu vượt 8000 tokens
 - LLM call với retry: 3 lần, backoff 1.5^n
 - Empty content + không có tool_calls → fallback
 - Tool execution với duplicate detection (tránh gọi tool trùng)
 - Guardrails: kiểm tra sensitive tool trước khi execute
4. **Loop Breaker:** ≥ 4 bước → fallback message kèm SĐT liên hệ

5.5 Timeout Protection

Layer	Timeout	Retries	Fallback
LLM call	360s	3×1.5^n backoff	Vui lòng liên hệ...
Tool execution	30s	0	Tool error message

5.6 Guardrails

src/system2/guardrails.py cung cấp bốn cơ chế bảo vệ:

1. **Tool Input Validation:** Kiểm tra schema tên và tham số tool.
2. **Sensitive Tool Approval:** Ba tool yêu cầu duyệt (send_zalo, send_sms, send_payment_reminder) → chuyển vào Approval Queue.
3. **Token Limit:** Giới hạn system + user + 8 messages gần nhất trong 8000 tokens.
4. **Loop Breaker:** max_iterations = 4.

```
SENSITIVE_TOOLS = {
    "send_payment_reminder": {
        "requires_approval": True,
        "approver_role": "landlord",
    },
    "send_zalo": {
        "requires_approval": True,
        "approver_role": "landlord",
    },
}
```

5.7 Multi-Turn Conversation

src/user_modeling/conversation_memory.py quản lý hội thoại đa lượt:

- ChatRequest.session_id: UUID tự sinh nếu client không cung cấp.
- Fetch 5 turns gần nhất, inject vào system prompt.
- Zalo: zalo-{sender_id}-{date} làm session ID (tự động reset mỗi ngày).
- DB-backed: conversation_history lưu JSONB tool_calls, metrics latency_ms, cost_usd.
- Cleanup: 02:00 daily, xóa turns > 30 ngày.

Tool Registry

6.1 Dynamic Loading

ToolRegistry (src/tools/tool_registry.py) chỉ load tools cần thiết cho intent, giảm context window:

```
INTENT_TOOLKIT_MAP = {
    "billing_inquiry": ["knowledge"],
    "payment_reminder": ["knowledge", "automation"],
    "maintenance_request": ["knowledge", "automation"],
    "room_recommendation": ["decision", "knowledge"],
    "contract_renewal": ["knowledge", "decision", "automation"],
    "background_event_*": ["knowledge", "automation"],
}
```

6.2 Decision Toolkit (5 tools)

Tool	Input	Output
fetch_available_rooms	budget_max, min_area, floor	Danh sách phòng trống
calc_rent	room_id, month, kwh, m3	Chi tiết tiền thuê
recommend_transfer	tenant_id	Gợi ý nâng cấp phòng
compare_rooms	room_id_1, room_id_2	So sánh song song
recommend_renewal	tenant_id	Mã trận tái ký 4 tín hiệu

recommend_renewal tính composite score:

$$\text{score} = 0.35 \times \text{payment} + 0.25 \times \text{cv} + 0.25 \times \text{churn_inv} + 0.15 \times \text{persona}$$

Phân loại: A (≥ 0.8): VIP Retain | B (≥ 0.6): Standard | C (≥ 0.4): Incentive | D (< 0.4): High Risk.

6.3 Knowledge Toolkit (7 tools)

Tool	Chức năng
get_invoice_detail	Chi tiết hóa đơn theo tháng
get_contract_status	Hợp đồng hiện tại (ngày còn lại, tiền cọc)
get_payment_history	Lịch sử N tháng (đúng hạn / trễ)
get_maintenance_status	Tra ticket bảo trì
get_room_info	Thông tin phòng (amenities JSONB)
query_policies	RAG trên knowledge_base/*.md
get_room_by_number	Guest-friendly (PII-safe)

6.4 Automation Toolkit (6 tools)

Tool	Chức năng	Độ nhạy
send_zalo	Gửi Zalo OA message	Cần duyệt
send_sms	Gửi SMS (max 160 ký tự)	Cần duyệt
create_maintenance_ticket	Tạo ticket bảo trì	—
send_payment_reminder	Tạo approval request	Cần duyệt
schedule_room_viewing	Đặt lịch xem phòng	—
update_user_preference	Cập nhật Core Memory	—

User Modeling & Personalisation

7.1 Kiến trúc

User Modeling Layer gồm 6 services:

```
User Modeling Layer
+-- ProfileService      user_profiles (tone, JSONB personalization)
+-- BehaviorTracker     behavior_logs (action_type, JSONB metadata)
+-- MemoryManager       user_embeddings (vector 3072-d, HNSW)
+-- ConversationMemory  conversation_history (JSONB tool_calls)
+-- PersonaOptimizer     trigger-based (flash_client)
+-- ApprovalService     approval_queue (pending/approved/rejected)
```

7.2 Persona Optimizer

src/user_modeling/persona_optimizer.py chạy theo **trigger** — chỉ xử lý tenant khi có hội thoại mới, dùng **flash model** (gemma-4-31b-it):

```
class PersonaOptimizer:
    def __init__(self, db_pool, llm_client):
        self.llm = llm_client

    async def optimize_tenant_profile(self, tenant_id):
        recent = await self._get_recent_conversations(
            tenant_id, since_last_run=True
        )
        if not recent:
            return None

        current_profile = await self._get_current_profile(tenant_id)
        prompt = (
            f'Current_profile: {current_profile}\n'
            f'Recent: {recent}\n'
            'Analyze and return updated profile.'
        )
        profile = await self.client.generate_structured(
            prompt, PersonalizationProfile
        )
        return profile
```

Ba quyết định thiết kế chính:

- **Flash model**: gemma-4-31b-it — đủ cho phân tích hành vi batch, chi phí thấp.

- **Trigger-based:** Chỉ chạy khi có hội thoại mới, tránh lãng phí API calls.
- **Full history:** Phân tích tất cả tương tác từ lần optimize cuối.

7.3 Core Memory Updates

Tool `update_user_preference` cho phép agent patch trực tiếp các keys hợp lệ trong `personalization_profile` JSONB:

```
valid_keys = {  
    "demographics": ["occupation", "age_group", "family_status"],  
    "personality_traits": ["openness", "conscientiousness"],  
    "preferences": ["communication_tone", "contact_method"],  
    "interaction_patterns": ["active_hours", "payment_habit"],  
}
```

7.4 Approval Service

`src/user_modeling/approval_service.py` (490 dòng) quản lý queue duyệt cho tác vụ nhạy cảm:

1. Tool gọi `create_request(tool_name, tool_args, tenant_id)` → INSERT `approval_queue` (status: pending)
2. LLM trả "Yêu cầu #{id} đang chờ quản lý duyệt"
3. Admin GET `/admin/approvals` → xem pending queue
4. Admin POST `/admin/approvals/{id}/approve` → status approved, execute action
5. Admin POST `/admin/approvals/{id}/reject` → status rejected

Proactive System

8.1 Bẫy Cron Jobs

src/cron/scheduler.py (481 dòng) quản lý bảy tác vụ định kỳ:

Job	Lịch	View	Điều kiện dispatch
Invoice overdue	09:00 daily	v_overdue_invoices	days_overdue $\in \{1, 3, 7, 14\}$
Payment due soon	08:00 daily	v_payment_due_soon	days_due $\in \{3, 1, 0\}$
Contract expiring	10:00 ngày 1	v_expiring_contracts	days_expiry $\in \{30, 14, 7, 1\}$
Maintenance	Mon 08:00	v_open_tickets	Priority SLA
Birthday	07:00 daily	v_upcoming_birthdays	days_until $\in \{3, 1, 0\}$
Persona opt.	Trigger	user_profiles	Hội thoại mới
Cleanup	02:00 daily	conversation_history	DELETE > 30 days

8.2 Event Dispatcher

8.3 Anti-Spam Guard

Mỗi cặp tenant-event được giới hạn configurable:

```
class AntiSpamGuard:
    def __init__(self, behavior_tracker, max_per_week=2):
        self.max_per_week = max_per_week

    async def can_send(self, tenant_id, event_type):
        count = await self.behaviors.count_recent_auto_actions(
            tenant_id, event_type, within_days=7
        )
        return count < self.max_per_week
```

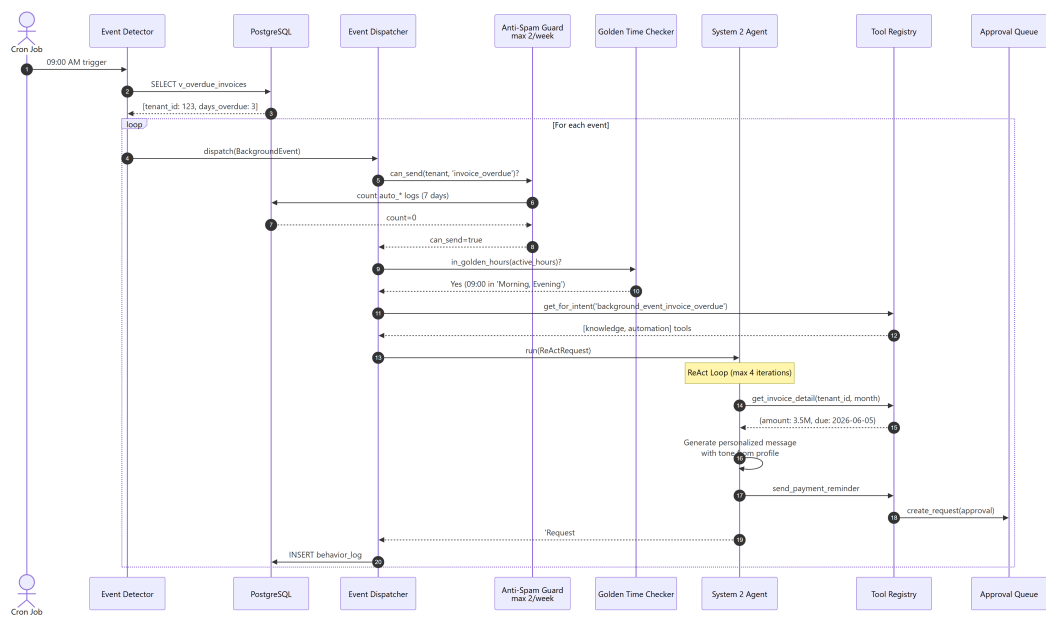


Figure 8.1: Event dispatcher cho proactive invoice reminder: Anti-Spam Guard kiểm tra tần suất, Golden Time Delivery tôn trọng khung giờ hoạt động của tenant, tool registry tạo nội dung, approval queue nếu cần.

8.4 Golden Time Delivery

EventDispatcher tôn trọng `active_hours` preference của tenant (ví dụ: "Sáng, Tối"). Nếu ngoài khung giờ, tự động delay qua APScheduler.

Database Schema

9.1 Entity Relationship Diagram

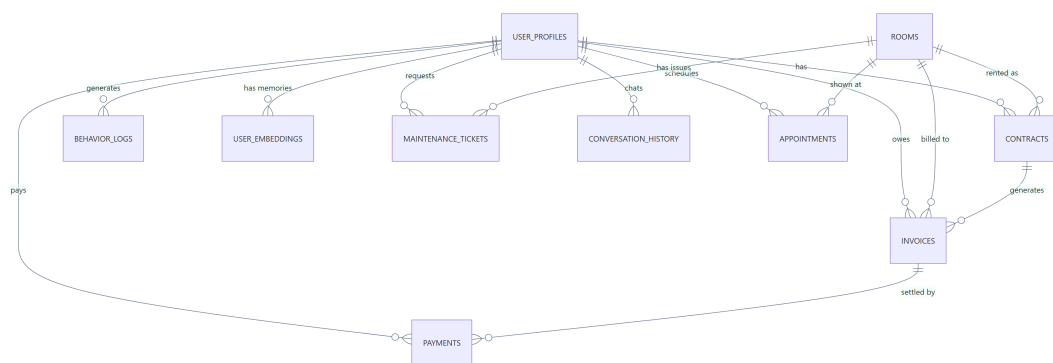


Figure 9.1: Sơ đồ quan hệ thực thể giữa 12 bảng của TroManager.

9.2 Danh sách bảng

Bảng	Mục đích	Trường chính
rooms	Phòng trọ	monthly_rent, amenities JSONB, status ENUM
user_profiles	Hồ sơ khách	full_name, phone, tone_preference ENUM
contracts	Hợp đồng	start_date, end_date, deposit_amount
invoices	Hóa đơn	invoice_month, total_amount, due_date
payments	Thanh toán	amount, payment_method, transaction_ref
behavior_logs	Log hành vi (PARTITIONED)	action_type, metadata JSONB
user_embeddings	Vector memory (HNSW)	embedding vector(3072), weight
semantic_cache	Cache ngữ nghĩa (HNSW)	query_embedding vector(3072)
maintenance_tickets	Ticket bảo trì	ticket_code (TKT-YYYY-XXXXXX)
conversation_history	Lịch sử chat	session_id UUID, tool_calls JSONB
approval_queue	Queue duyệt	tool_name, tool_args JSONB
appointments	Lịch hẹn	scheduled_at, status ENUM

9.3 SQL Views & Indexing

Bây SQL views hỗ trợ truy vấn thường gặp: v_overdue_invoices, v_payment_due_soon, v_expiring_contracts, v_upcoming_birthdays, v_open_maintenance_tickets, v_tenant_behavior_90d, v_active_tenants.

Chiến lược indexing:

- **Vector:** HNSW (vector_cosine_ops) trên user_embeddings và semantic_cache
- **PARTITION:** behavior_logs range-partitioned theo tháng
- **Composite:** behavior_logs(tenant_id, action_type, timestamp DESC)

Observability & API

10.1 JSON Structured Logging

Mọi log output dạng JSON với auto-redact credentials:

```
{
  "timestamp": "2026-06-14T09:00:00.123Z",
  "level": "INFO",
  "logger": "troManager.react_agent",
  "message": "Route decision",
  "request_id": "abc-123-def"
}
```

Các field được redact tự động: password, api_key, token, DB_PASSWORD, GEMINI_API_KEY.

10.2 Prometheus Metrics

GET /metrics (Prometheus text format, hỗ trợ ?format=json). Mười lăm metrics pre-registered:

- http_requests_total: Tổng số request HTTP (path normalized)
- llm_calls_total, llm_tokens_total: Chi phí LLM
- tool_calls_total: Số lần gọi tool
- system1_requests, system2_requests: Phân bố System 1/2
- cron_jobs_total, notifications_sent: Cron và notification

10.3 API Endpoints

Endpoint	Method	Auth	Chức năng
POST /chat	Public	—	Route → S1/S2
POST /webhook/zalo	Public	HMAC-SHA256	Zalo webhook
GET /metrics	Public	—	Prometheus metrics
GET /admin/approvals	Admin	X-Admin-Key	Queue duyệt
POST .../{id}/approve	Admin	X-Admin-Key	Duyệt + execute
POST .../{id}/reject	Admin	X-Admin-Key	Từ chối
GET/POST/DELETE /api/kb/*	Admin	X-Admin-Key	Knowledge Base CRUD

10.4 Bảo mật

- **Zalo Webhook:** HMAC-SHA256, constant-time compare (`hmac.compare_digest`)
- **Admin API:** X-Admin-Key header, so sánh constant-time
- **Tool-level auth:** `current_tenant_id` contextvar ngăn cross-tenant data access
- **PII protection:** `get_room_by_number` loại bỏ tenant info khi guest query
- **Rate limiting:** 60 req/min (tenant), 1000 req/h (IP)

Cấu hình & Triển khai

11.1 Hệ thống Config hai tầng

```
config/
+-- config.yaml           -- Main config (versioned)
+-- llm_config.yaml       -- LLM defaults (versioned)
+-- llm_config.local.yaml -- Secrets (gitignored)
+-- notifications.yaml    -- Zalo/SMS config
```

LLM config sử dụng biến môi trường cho API keys:

```
llm:
  provider: "openai_compatible"
  base_url: "https://generativelanguage.googleapis.com/v1beta/openai/"
  api_key: "${GEMINI_API_KEY}"
  flash_model: "gemma-4-31b-it"
  pro_model: "gemini-3.1-flash-lite"
  router_model: "gemma-4-26b-a4b-it"
  strip_thought: true
  request_timeout: 60
  default_max_tokens: 8192
```

11.2 Quick Start

```
conda create -n tromanager python=3.13
conda activate tromanager
pip install -r requirements.txt

cp .env.example .env # Fill GEMINI_API_KEY, DB_PASSWORD, ...

psql -h localhost -U postgres -d tromanager_db -f database/schema.sql
psql -h localhost -U postgres -d tromanager_db -f database/seed_data.sql

python scripts/run_smoke_test.py
python -m uvicorn src.main:app --host 0.0.0.0 --port 8000
```


Đánh giá

12.1 Kịch bản đánh giá

Để đánh giá hiệu quả của hệ thống, 20 kịch bản mô phỏng được xây dựng dựa trên dữ liệu seed thực tế (5 tenants, 9 rooms). Các kịch bản được chia làm hai nhóm:

- **Khách đã thuê (10 kịch bản):** Mỗi kịch bản là hội thoại đa lượt (4 turns) bao gồm tra cứu hóa đơn, báo bảo trì, gia hạn hợp đồng, khiếu nại, chuyển phòng, chấm dứt hợp đồng.
- **Khách mới (10 kịch bản):** Mỗi kịch bản một lượt, mô phỏng tìm phòng, so sánh phòng, hỏi thủ tục, đặt lịch xem phòng.

12.2 Kết quả

Loại	Số kịch bản	PASS	Nội dung
Khách đã thuê (multi-turn)	10	10/10	Tra cứu hóa đơn, bảo trì, gia hạn, khiếu nại, chuyển phòng, chấm dứt hợp đồng
Khách mới (single-turn)	10	10/10	Tìm phòng, so sánh, thủ tục, đặt lịch, an ninh, tính chi phí

Tất cả 20 kịch bản đều pass, bao gồm các tình huống:

- Tra cứu hóa đơn với tenant đã thanh toán và tenant quá hạn
- Bảo trì với ticket code thực tế (TKT-2026-0002, TKT-0003)
- Khiếu nại tiếng ồn với chính sách giờ yên tĩnh
- Chuyển phòng từ 201 lên 202 (ban công rộng)
- Gia hạn hợp đồng với thương lượng giá
- Chấm dứt hợp đồng sớm với thương lượng phí phạt

12.3 Phân tích độ trễ

Mỗi turn hội thoại có độ trễ ~6 giây (do rate limiting trong test). Trong thực tế, System 1 phản hồi trong 1–3 giây, System 2 trong 5–15 giây tùy độ phức tạp.

Tổng kết & Lộ trình

13.1 Tổng kết

Báo cáo trình bày **TroManager Architecture #2**, một hệ thống AI đa tác tử cho quản lý nhà trọ với kiến trúc Router-Centric Dual-Process. Hệ thống kết hợp hai luồng xử lý: System 1 cho câu hỏi nhanh (semantic cache + flash LLM) và System 2 cho tác vụ phức tạp (ReAct agent + tool registry). Bảy cron jobs chủ động, cơ chế Human-in-the-loop qua Approval Queue, và Persona Optimizer trigger-based hoàn thiện hệ thống.

13.1.1 Các luận điểm chính

1. Kiến trúc Dual-Process với LLM Intent Router cho phép phân tách chi phí hiệu quả: System 1 xử lý 80% câu hỏi với chi phí thấp.
2. Semantic Cache trên pgvector (cosine >0.9) loại bỏ hoàn toàn LLM call cho các câu hỏi lặp lại.
3. Dynamic Tool Registry giảm context window cho agent bằng cách chỉ load tools cần thiết.
4. Cron jobs chủ động với Anti-Spam Guard và Golden Time Delivery đảm bảo tương tác không gây phiền nhiễu.
5. Trigger-based Persona Optimizer tiết kiệm chi phí bằng cách chỉ chạy khi có dữ liệu mới.

13.1.2 Hiện trạng

Hạng mục	Trạng thái
Dual-Process Architecture (Router → S1/S2)	Đã triển khai
18 tools dynamic loading (3 toolkits)	Kết nối DB thực
Semantic Cache (pgvector, cosine >0.9)	Cache warming với FAQ
ReAct Agent (4 iterations, 3 retries)	Timeout + dupe protection
Guardrails + Approval Queue	Human-in-the-loop
7 Cron Jobs (invoice, contract, birthday)	Golden Time + Anti-Spam
Persona Optimizer (trigger-based)	Pydantic structured output
Multi-turn Conversation	5-turn context
Zalo Webhook (HMAC-SHA256)	Dev skip mode
Knowledge Base CRUD API	Auto-reload RAG
Prometheus Metrics	15+ metrics
JSON Structured Logging	Auto-redact + Request ID
Graceful Shutdown	Drain + timeout

13.2 Lộ trình tương lai

Phase	Mục tiêu
Observability	Grafana dashboard, cost tracking theo tenant
Phase 2	Multi-Branch agents, RBAC, GDPR compliance
Phase 3	Knowledge Graph, Multi-Agent coordination, Voice support
Phase 4	Admin UI (Next.js), real-time dashboard

13.3 Key Source Files

File	Dòng	Mô tả
src/main.py	1228	FastAPI app + service wiring
src/system2/react_agent.py	518	Core ReAct Agent
src/tools/decision_tools.py	536	5 decision tools
src/tools/knowledge_tools.py	502	7 knowledge tools
src/tools/automation_tools.py	448	6 automation tools
src/system1/fast_layer.py	337	System 1 pipeline
src/gateway/keyword_classifier.py	113	LLMIntent Router
src/gateway/router.py	165	Router Gateway
src/cron/scheduler.py	481	7 cron jobs
src/user_modeling/approval_service.py	490	Approval queue
src/user_modeling/persona_optimizer.py	155	Persona Optimizer
src/system2/guardrails.py	191	Guardrails
src/system1/semantic_cache.py	166	Semantic Cache
database/schema.sql	552	Full PostgreSQL schema